

Версия файла	Кто изменил	Что изменено	Время
0.1	Воронина	Заполнены разделы 1, 2 Накиданы 3.1	2026-05-26-04:12
0.2	Коновалов	Заполнен раздел 3, 4.1	2026-05-27 14:21
1.0	Коновалов, Воронина	Заполнены все разделы, зафиксирована версия для сдачи	2026-06-03 16:42

Software Development Plan (План разработки продукта)

1. Introduction (Введение)

1.1 Purpose

Назначение данного документа - определить план разработки системы PollosFlow, а именно состав работ, организацию команды, план фаз и релизов, необходимые ресурсы, бюджет, механизмы контроля сроков, качества, требований и рисков. Документ используется как управленческая основа проекта и как соглашение между командой разработки и заказчиком о том, каким образом будет организована разработка системы.

Документ переводит требования к продукту в план работ, ресурсов и контрольных точек. Особое внимание уделяется рискам, связанным с микросервисной архитектурой, интеграциями с POS и банковскими сервисами, безопасностью доступа, производительностью и кадровой устойчивостью команды.

1.2 Scope (Область применения)

Данный документ относится к проекту PollosFlow – системе легализации денежных средств и управления финансовыми потоками сети ресторанов быстрого питания Los Pollos Hermanos. Система предназначена для регистрации поступлений, планирования распределения средств по ресторанам, генерации кассовых операций и дальнейшего распределения денежных средств.

Документ будет использоваться заказчиком, руководителем проекта, аналитиками, разработчиками, тестировщиками, DevOps-инженером и ответственными за эксплуатацию. План охватывает работы от уточнения требований до передачи системы в эксплуатацию.

1.4 Definitions, Acronyms and Abbreviations (Определения и аббревиатуры)

[Glossary](#)

1.5 Overview (Обзор документа)

Раздел 2 содержит обзор проекта, цели, ограничения и ожидаемые результаты.

Раздел 3 описывает организацию команды, роли и внешние интерфейсы взаимодействия. Раздел 4

определяет управленческий процесс: оценки сроков и бюджета, фазовый план, расписание, ресурсы, контроль требований, сроков, бюджета, качества, отчётности и рисков. Завершающий подраздел описывает критерии закрытия проекта и передачу системы в эксплуатацию.

2. Project Overview (Обзор продукта)

2.1 Project Purpose, Scope, and Objectives (Назначение, цели и контекст продукта)

Проект направлен на создание закрытой информационной системы для автоматизации управления финансовыми потоками сети ресторанов Los Pollos Hermanos. Система предназначена для централизованного учёта поступлений, планирования распределения средств между ресторанами сети, контроля кассовых операций, взаимодействия с внешними системами, генерации отчетности и мониторинга финансового состояния.

Цель проекта – автоматизировать процесс легализации денежных средств и управления финансовыми потоками сети Los Pollos Hermanos.

Контекст продукта предполагает эксплуатацию системы в условиях повышенных требований к конфиденциальности, устойчивости к сбоям, контролю доступа и сохранности данных.

2.2 Assumptions and Constraints (Влияющие факторы и ограничения)

1. Срок проекта ограничен одним учебным семестром.
2. Проект имеет кадровое ограничение, команда разработки формируется из ограниченного числа доверенных специалистов. Из-за закрытого характера деятельности Los Pollos Hermanos привлечение внешних специалистов ограничено.
3. Проект зависит от наличия тестовой POS-системы.
4. ~~Бюджет не ограничен))) уже наотмывали \$~~

2.3 Project Deliverables (Ожидаемые результаты проекта)

1. Информационная система PollosFlow
2. Эмулятор POS-системы
3. Эмулятор банковской системы
4. Генератор данных для business baseline – инструмент для создания тестовых данных по ресторанам: меню, средний чек, поток клиентов, сезонность, кассовые операции и смены.
5. Модуль расчёта распределения по ресторанам с учётом baseline, допустимых отклонений, нагрузки точки и заданных ограничений.
6. Банковские red flags – набор правил, определяющих операции, которые банковская система может считать подозрительными: превышение лимитов, резкие отклонения оборота, частые однотипные операции, несоответствие baseline и расхождения с кассовыми данными.
7. Модуль генерации отчетности – формирование отчетов по операциям, партиям средств, ресторанам, сверке, Z-отчетам и экспорт в PDF/Excel/CSV.
8. База данных – структура PostgreSQL для хранения пользователей, ролей, операций, партий средств, ресторанов, baseline, отчетов, аудита и статусов обработки.
9. API-документация – описание REST API и контрактов между микросервисами.

2.4 Evolution of the Software Development Plan (План развития данного документа)

Версия **0.1** создаётся на этапе инициации проекта. В нее включаются цель проекта, область применения, предварительные ограничения, ожидаемые результаты и базовое описание команды.

Версия **0.2** создается после утверждения SRS. В нее добавляются уточненный состав работ, список модулей PollosFlow, технологический стек, зависимости от POS-эмулятора, банковского эмулятора, baseline-генератора и инфраструктурных компонентов.

Версия **0.3** создаётся после завершения проектирования архитектуры. В ней уточняются микросервисы, MFE-модули, API-контракты, схема базы данных, план интеграции, тестовые стенды и распределение ответственности между участниками команды.

Версия **0.4** создаётся перед началом реализации. В неё вносится финальное расписание работ на семестр, состав релизов, план тестирования, план ресурсов и бюджет проекта.

Версия **0.5** создаётся перед интеграционным тестированием. В ней уточняются сценарии проверки основного процесса.

Версия **1.0** создаётся перед итоговой сдачей проекта. Она фиксирует фактически выполненный объём работ, итоговый состав релиза, реализованные модули, оставшиеся ограничения, результаты тестирования и финальный план завершения проекта.

Документ должен быть изменен при изменении SRS, сроков семестра, состава команды, бюджета, технологического стека, состава релиза, архитектуры системы, внешних зависимостей или требований к тестированию.

Ответственным за актуальность SDP является руководитель (Сене4ка) проекта. Изменения в документе согласуются с командой и фиксируются в истории версий.

3. Project Organization (Организация проекта)

3.1 Organizational Structure (Структура команды разработки)

Команда разработки проекта PollosFlow состоит из трёх участников: Сенечка, Дора и П(онаприехавший)етя. Из-за ограниченного размера команды каждый участник совмещает несколько ролей.

Сеня выполняет роль руководителя проекта, backend-разработчика и DevOps-инженера. Он отвечает за планирование работ, контроль сроков, реализацию серверной части системы, базы данных PostgreSQL, RabbitMQ, Redis, Keycloak, а также за подготовку окружения для запуска и тестирования системы.

Дора выполняет роль системного аналитика, frontend-разработчика и UI/UX-ответственного. Она отвечает за уточнение требований, описание пользовательских сценариев, проектирование интерфейсов для ролей владельца, администратора, бухгалтера, кассира и авторизованного пользователя, а также за реализацию клиентской части системы.

Петя выполняет роль backend-разработчика, QA-инженера, интеграционного разработчика и ответственного за тестовые эмуляторы. Он отвечает за подготовку тест-кейсов, проверку функциональности, тестирование ролей доступа, реализацию и проверку POS-эмулятора,

банковского эмулятора, генератора baseline-данных, а также за интеграционное тестирование сквозного сценария системы.

При необходимости участники команды взаимно проверяют код, документацию и результаты друг друга.

3.2 External Interfaces (Внешние интерфейсы)

Команда проекта PollosFlow взаимодействует с внешними участниками через ограниченный набор каналов связи. Внешнее взаимодействие необходимо для уточнения требований, согласования проектных решений, получения обратной связи по предметной области, проверки промежуточных результатов и подготовки итоговой сдачи проекта.

Основным внешним представителем заказчика является владелец сети Los Pollos Hermanos. Он отвечает за согласование бизнес-целей проекта, подтверждение требований к функциональности(!) системы, утверждение логики распределения средств, baseline-показателей ресторанов, состава отчетности и правил обработки спорных операций. Взаимодействие с владельцем осуществляется через регулярные встречи, текстовые согласования и демонстрации промежуточных результатов.

Дополнительным внешним контактным лицом со стороны предметной области является бухгалтер сети. Он участвует в уточнении требований к финансовой отчетности, сверке данных, Z-отчетам, банковским выпискам и экспортируемым документам. Бухгалтер предоставляет обратную связь по корректности отчетных форм, структуре финансовых данных и требованиям к согласованности операций.

Каналы взаимодействия между командой разработки и внешними участниками определяются следующим образом:

Участник / интерфейс	Назначение взаимодействия	Канал связи	Ответственный со стороны команды
Владелец сети	Согласование целей, требований, планов распределения, baseline и критических решений	Встречи, текстовые согласования, демонстрации	Сеня
Бухгалтер	Уточнение требований к отчетности, сверке, финансовым данным и экспортам	Встречи, комментарии к документам, демонстрации	Дора, Сеня
POS-эмулятор	Проверка передачи операций, закрытия смен, формирования Z-отчетов	REST API / тестовый стенд	Петя

Банковский эмулятор	Проверка банковских операций, выписок и сверки данных	REST API / тестовый стенд	Петя
---------------------	---	---------------------------	------

Для технического взаимодействия используются следующие интерфейсы:

- репозиторий исходного кода – для хранения кода, ревью изменений и фиксации истории разработки;
- таск-трекер – для постановки задач, контроля статусов и распределения ответственности;
- документация проекта – для хранения требований, архитектурных решений, сценариев использования и планов разработки;
- тестовый стенд – для проверки работы микросервисов, frontend-модулей, POS-эмулятора, банковского эмулятора и интеграционных сценариев;
- API-документация OpenAPI / Swagger – для согласования контрактов между сервисами и тестовыми внешними системами.

Внешние интерфейсы должны использоваться таким образом, чтобы все критически важные решения были зафиксированы письменно. Устные договоренности не считаются основанием для изменения требований или плана разработки до тех пор, пока они не внесены в соответствующий документ проекта.

3.3 Roles and Responsibilities (Роли и обязанности)

3.3.1 Руководитель проекта

Руководитель проекта отвечает за организацию разработки, планирование работ, контроль сроков, управление изменениями и координацию участников команды. В рамках проекта данную роль выполняет Сеня.

Основные обязанности руководителя проекта:

1. формирование и актуализация плана разработки;
2. контроль выполнения задач по фазам и итерациям;
3. распределение задач между участниками команды;
4. контроль сроков выполнения работ;
5. согласование изменений требований и состава релиза;
6. контроль рисков проекта;
7. организация демонстраций промежуточных результатов;
8. подготовка проекта к итоговой сдаче;
9. коммуникация с преподавателем, заказчиком и внешними участниками.

3.3.2 Системный аналитик

Системный аналитик отвечает за уточнение требований, описание пользовательских сценариев, согласование функциональности и поддержку связности проектной документации. В рамках проекта данную роль выполняет Дора.

Основные обязанности системного аналитика:

1. уточнение требований к системе;
2. поддержка SRS в актуальном состоянии;
3. описание пользовательских ролей и сценариев использования;
4. подготовка use-case диаграммы;
5. согласование требований к интерфейсам;
6. описание ограничений, допущений и зависимостей;
7. участие в анализе рисков;
8. проверка соответствия реализуемой функциональности требованиям.

3.3.3 Backend-разработчик

Backend-разработчик отвечает за реализацию серверной части системы, бизнес-логики, микросервисов, API, работы с базой данных и интеграций. В рамках проекта данную роль выполняют Сеня и Петя.

Основные обязанности backend-разработчика:

1. проектирование и реализация серверных модулей;
2. реализация REST API;
3. разработка логики регистрации поступлений, партий средств и статусов обработки;
4. реализация алгоритмов распределения и работы с business baseline;
5. реализация интеграции с PostgreSQL, RabbitMQ и Keycloak;
6. реализация модулей отчетности, сверки и аудита;
7. обработка ошибок и отказов внешних зависимостей;
8. участие в code review;
9. исправление дефектов, выявленных при тестировании.

3.3.4 Frontend-разработчик

Frontend-разработчик отвечает за реализацию клиентской части системы и пользовательских интерфейсов для всех ролей. В рамках проекта данную роль выполняет Дора.

Основные обязанности frontend-разработчика:

1. реализация MFE-модулей клиентской части;
2. разработка интерфейсов для владельца, администратора, бухгалтера, кассира и авторизованного пользователя;
3. реализация форм ввода, таблиц, фильтров, дашбордов и уведомлений;
4. обеспечение ролевого отображения интерфейса;
5. реализация валидации данных на уровне UI;
6. поддержка единого визуального стиля интерфейса;
7. обеспечение адаптивности интерфейса для ПК и планшетов;
8. участие в проверке пользовательских сценариев.

3.3.5 UI/UX-ответственный

UI/UX-ответственный отвечает за удобство использования системы, консистентность интерфейсов и соответствие UI-паттернам, указанным в SRS. В рамках проекта данную роль выполняет Дора.

Основные обязанности UI/UX-ответственного:

1. проектирование структуры пользовательских экранов;
2. подготовка интерфейсных макетов;
3. контроль единообразия интерфейса;
4. применение выбранных UI-паттернов: Dashboard, Sidebar Navigation, Data Table, Form, Wizard, Modal Window, Notification Pattern;
5. проверка понятности интерфейса для разных уровней пользователей;
6. минимизация количества действий для типовых операций;
7. проектирование визуальной обратной связи, статусов и сообщений об ошибках.

3.3.6 DevOps-инженер

DevOps-инженер отвечает за подготовку окружения, запуск сервисов, настройку инфраструктурных компонентов и обеспечение воспроизводимости развертывания. В рамках проекта данную роль выполняет Сенья.

Основные обязанности DevOps-инженера:

1. настройка окружения разработки и тестирования;
2. подготовка конфигураций для запуска микросервисов;
3. настройка PostgreSQL, RabbitMQ, Redis, Keycloak, Logstash и Sentry;
4. обеспечение сборки и запуска проекта;
5. настройка переменных окружения;
6. поддержка тестового стенда;
7. контроль логирования и мониторинга;
8. устранение инфраструктурных проблем во время разработки и тестирования.

3.3.7 QA-инженер

QA-инженер отвечает за проверку функциональности, подготовку тест-кейсов, проведение тестирования и фиксацию дефектов. В рамках проекта данную роль выполняет Петя.

Основные обязанности QA-инженера:

1. подготовка тест-кейсов по требованиям SRS;
2. проверка основных пользовательских сценариев;
3. тестирование ролевой модели доступа;
4. проверка корректности форм, валидации и сообщений об ошибках;
5. тестирование интеграций с POS-эмулятором и банковским эмулятором;
6. проведение интеграционного тестирования сквозного сценария;
7. фиксация дефектов и контроль их исправления;
8. участие в приемочном тестировании.

3.3.8 Интеграционный разработчик

Интеграционный разработчик отвечает за реализацию и проверку взаимодействия PollosFlow с внешними и внутренними сервисами. В рамках проекта данную роль выполняет Петя.

Основные обязанности интеграционного разработчика:

1. разработка POS-эмулятора;
2. разработка банковского эмулятора;

3. согласование API-контрактов;
4. проверка обмена данными между микросервисами;
5. тестирование очередей сообщений RabbitMQ;
6. проверка retry, timeout и fallback-сценариев;
7. тестирование передачи операций в POS-систему;
8. тестирование получения банковских выписок и сверки данных.

3.3.9 Ответственный за документацию

Ответственный за документацию обеспечивает актуальность проектных документов и согласованность между Vision, SRS, SDP, use-case диаграммы и risk management plan. Данная ответственность распределяется между всеми участниками команды, при этом финальный контроль выполняет руководитель проекта.

Основные обязанности:

1. поддержка актуальности SDP;
2. внесение изменений в SRS при изменении требований;
3. оформление use-case диаграммы;
4. поддержка risk management plan;
5. фиксация изменений в истории версий;
6. проверка соответствия документов друг другу;
7. подготовка документации к итоговой сдаче.

Матрица ответственности:

Область работ	Сеня	Дора	Петя
Управление проектом	Ответственный	Участвует	Участвует
SRS и требования	Участвует	Ответственная	Участвует
SDP	Ответственный	Участвует	Участвует
Backend	Ответственный	Не участвует	Участвует
Frontend	Участвует	Ответственная	Не участвует

UI/UX	Участвует	Ответственная	Участвует в проверке
DevOps и инфраструктура	Ответственный	Не участвует	Участвует
POS-эмулятор	Участвует	Не участвует	Ответственный
Банковский эмулятор	Участвует	Не участвует	Ответственный
QA и тестирование	Участвует	Участвует	Ответственный
Интеграционное тестирование	Участвует	Участвует	Ответственный
Документация	Ответственный за финальную сборку	Ответственная за требования и UI	Ответственный за тесты и интеграции

4. Management Process (Процесс управление)

4.1 Project Estimates (Оценка сроков разработки проекта)

Наш прекрасный Гантт - <https://app.ganttpro.com/#/project/1780490065857/gantt>

Проект реализуется в два этапа:

- первый с 09.02.2026 по 29.04.2026
- второй с 01.09.2026 по 30.11.2026

Оценка трудозатрат по видам работ:

Категория работ	Минимальное время	Ожидаемое время	Максимальное время
-----------------	-------------------	-----------------	--------------------

Анализ требований и документация	70 ч	90 ч	115 ч
Проектирование архитектуры	60 ч	80 ч	105 ч
Backend-разработка	120 ч	150 ч	190 ч
Frontend-разработка	85 ч	110 ч	140 ч
Эмуляторы POS и банковской системы	55 ч	70 ч	95 ч
Интеграция и DevOps	40 ч	50 ч	70 ч
Тестирование	45 ч	50 ч	75 ч
Исправление дефектов и стабилизация	20 ч	20 ч	25 ч
Итого	495 ч	620 ч	815 ч

Рассчитаем ориентировочное количество временных затрат по участникам

Участник	Минимальное время	Ожидаемое время	Максимальное время
Сеня	200 ч	250 ч	320 ч
Дора	150 ч	190 ч	245 ч
Петя	145 ч	180 ч	240 ч

Итого	495 ч	620 ч	815 ч
--------------	--------------	--------------	--------------

Составим таблицу с расчетом цены проекта:

Участник	Минимальная стоимость	Ожидаемая стоимость	Максимальная стоимость
Сеня	816 500 руб.	1 017 000 руб.	1 297 500 руб.
Дора	468 000 руб.	592 500 руб.	764 500 руб.
Петя	496 000 руб.	618 000 руб.	825 000 руб.
Итого	1 780 500 руб.	2 227 500 руб.	2 887 000 руб.

Сеня:

Роль	Ставка	Мин время	Мин стоимость	Ожидаемое время	Ожидаемая стоимость	Макс время	Макс стоимость
Руководитель проекта	4 600 руб./ч	75 ч	345 000 руб.	90 ч	414 000 руб.	110 ч	506 000 руб.
Backend-разработчик	3 800 руб./ч	90 ч	342 000 руб.	110 ч	418 000 руб.	145 ч	551 000 руб.
DevOps-инженер	3 700 руб./ч	35 ч	129 500 руб.	50 ч	185 000 руб.	65 ч	240 500 руб.
Итого		200 ч	816 500 руб.	250 ч	1 017 000 руб.	320 ч	1 297 500 руб.

Дора:

Роль	Ставка	Мин время	Мин стоимость	Ожидаемое время	Ожидаемая стоимость	Макс время	Макс стоимость
Системный аналитик	3 400 руб./ч	45 ч	153 000 руб.	55 ч	187 000 руб.	70 ч	238 000 руб.
Frontend-разработчик	3 200 руб./ч	60 ч	192 000 руб.	80 ч	256 000 руб.	105 ч	336 000 руб.
UI/UX-ответственный	2 900 руб./ч	30 ч	87 000 руб.	35 ч	101 500 руб.	45 ч	130 500 руб.
Ответственный за документацию	2 400 руб./ч	15 ч	36 000 руб.	20 ч	48 000 руб.	25 ч	60 000 руб.
Итого		150 ч	468 000 руб.	190 ч	592 500 руб.	245 ч	764 500 руб.

Петя:

Роль	Ставка	Мин время	Мин стоимость	Ожидаемое время	Ожидаемая стоимость	Макс время	Макс стоимость
Backend-разработчик	3 800 руб./ч	35 ч	133 000 руб.	45 ч	171 000 руб.	60 ч	228 000 руб.
Интеграционный разработчик	3 600 руб./ч	55 ч	198 000 руб.	70 ч	252 000 руб.	95 ч	342 000 руб.
QA-инженер	3 000 руб./ч	55 ч	165 000 руб.	65 ч	195 000 руб.	85 ч	255 000 руб.

Итого		145 ч	496 000 руб.	180 ч	618 000 руб.	240 ч	825 000 руб.
--------------	--	--------------	---------------------	--------------	---------------------	--------------	---------------------

4.2 Project Plan (План проекта)

4.2.1 Phase Plan (План фаз)

Фаза	Количество итераций	Начало	Конец
Фаза 1. Инициация проекта	1	2026-02-09	2026-02-11
Фаза 2. Анализ требований	1	2026-02-12	2026-02-27
Фаза 3. Проектирование архитектуры	1	2026-03-02	2026-03-13
Фаза 4. Реализация основных модулей	3	2026-03-16	2026-04-24
Фаза 5. Интеграция и сквозные сценарии	2	2026-04-27	2026-09-25
Фаза 6. Тестирование и стабилизация	2	2026-09-28	2026-10-23
Фаза 7. Подготовка итогового релиза	1	2026-10-26	2026-11-13

4.2.2 Iteration Objectives (Цели итераций)

Фаза	Номер итерации	Описание	Вехи
Фаза 1. Инициация проекта	1.1	Определение цели проекта, границ системы PollosFlow, состава команды, ограничений, ожидаемых результатов и предварительного состава поставляемых артефактов.	Утверждена цель проекта; определена область применения; определен состав команды; зафиксированы ограничения проекта; подготовлена начальная версия SDP; определен предварительный список deliverables.

Фаза 2. Анализ требований	2.1	Уточнение функциональных и нефункциональных требований, описание пользовательских ролей, зависимостей, ограничений и основных сценариев использования.	Подготовлен SRS; описаны пользовательские роли; описаны функциональные требования; подготовлены use-case диаграммы; подготовлены описания прецедентов; сформирован первичный Risk List.
Фаза 3. Проектирование архитектуры	3.1	Проектирование микросервисной архитектуры серверной части, MFE-клиента, структуры базы данных, API-контрактов и интеграций с POS-эмулятором и банковским эмулятором.	Подготовлена архитектурная схема; определен состав микросервисов; определены MFE-модули; спроектирована структура PostgreSQL; описаны REST API-контракты; подготовлен план интеграции.
Фаза 4. Реализация основных модулей	4.1	Реализация базовой серверной части, управления пользователями, ролевой модели, аутентификации/авторизации, начальных frontend-модулей и основной структуры интерфейса.	Реализованы базовые backend-сервисы; настроена работа с пользователями и ролями; подготовлены начальные MFE-интерфейсы; реализованы базовые экраны входа и ролевой навигации; подготовлено окружение для дальнейшей реализации.
Фаза 4. Реализация основных модулей	4.2	Реализация модулей партий средств, business baseline, планирования распределения и связанных пользовательских экранов.	Реализован учет партий средств; реализовано управление baseline-показателями ресторанов; реализован модуль планирования распределения; реализованы интерфейсы просмотра партий, baseline и планов распределения.

Фаза 4. Реализация основных модулей	4.3	Реализация базовой отчетности, аудита действий, обработки статусов операций, генерации кассовых операций и доработка backend/frontend-модулей.	Реализована генерация кассовых операций; реализован журнал аудита; реализована базовая отчетность; реализовано отображение статусов операций; выполнена доработка основных backend- и frontend-компонентов.
Фаза 5. Интеграция и сквозные сценарии	5.1	Подключение POS-эмулятора и проверка сценариев передачи кассовых операций, закрытия смен и формирования Z-отчетов.	Подключен POS-эмулятор; проверена передача операций в POS; проверено закрытие смены; проверено формирование Z-отчета; зафиксированы ошибки интеграции с POS.
Фаза 5. Интеграция и сквозные сценарии	5.2	Подключение банковского эмулятора, проверка банковских выписок, сверки данных и сквозного сценария обработки партии средств.	Подключен банковский эмулятор; проверено получение банковских выписок; проверена сверка внутренних данных с банковскими данными; проверен сквозной сценарий «приём → планирование → POS → бухгалтерия → банк».
Фаза 6. Тестирование и стабилизация	6.1	Проведение функционального, интеграционного и ролевого тестирования. Проверка основных пользовательских сценариев и фиксация выявленных дефектов.	Подготовлены тест-кейсы; выполнено функциональное тестирование; выполнено интеграционное тестирование; проверена ролевая модель доступа; дефекты зафиксированы в списке ошибок.
Фаза 6. Тестирование и стабилизация	6.2	Проведение регрессионного тестирования, исправление критических дефектов, стабилизация системы и проверка соответствия требованиям SRS.	Исправлены дефекты уровней Critical и Major; выполнено регрессионное тестирование; подтверждена работоспособность основных сценариев; проверено соответствие ключевым требованиям SRS.

Фаза 7. Подготовка итогового релиза	7.1	Финальная сборка проекта, обновление документации, подготовка демонстрационного сценария и передача проекта к итоговой сдаче.	Подготовлен итоговый релиз; обновлены SDP, SRS, use-case диаграммы и Risk List; подготовлен демонстрационный сценарий; проект готов к итоговой сдаче.
---	-----	---	---

4.2.3 Releases (Релизы)

Номер версии	Описание релиза	Дата выпуска
0.1	Начальный управленческий релиз проекта. Включает определение цели проекта, области применения, состава команды, ограничений, ожидаемых результатов и предварительного состава поставляемых артефактов.	2026-02-11
0.2	Релиз требований. Включает подготовленный SRS, описание пользовательских ролей, функциональные и нефункциональные требования, use-case диаграммы и первичный Risk List.	2026-02-27
0.3	Архитектурный релиз. Включает архитектурную схему системы, состав микросервисов, MFE-модулей, структуру PostgreSQL, API-контракты и план интеграции с POS-эмулятором и банковским эмулятором.	2026-03-13
0.4	Релиз базовой реализации. Включает базовые backend-сервисы, ролевую модель, аутентификацию/авторизацию, начальные MFE-интерфейсы и подготовленное окружение разработки.	2026-04-03
0.5	Релиз основных бизнес-модулей. Включает модули партий средств, business baseline, планирования распределения, генерации кассовых операций, базовой отчетности, аудита и отображения статусов операций.	2026-04-24
0.6	Интеграционный релиз POS. Включает подключенный POS-эмулятор, проверку передачи кассовых операций, закрытия смен и формирования Z-отчетов.	2026-09-11
0.7	Интеграционный релиз банковского контура. Включает подключенный банковский эмулятор, получение банковских выписок, сверку данных и проверку сквозного сценария обработки партии средств.	2026-09-25

0.8	Тестовый релиз. Включает результаты функционального, интеграционного и ролевого тестирования, список выявленных дефектов и исправление основных ошибок.	2026-10-09
0.9	Стабилизационный релиз. Включает исправление дефектов уровней Critical и Major, результаты регрессионного тестирования и подтверждение соответствия ключевым требованиям SRS.	2026-10-23
1.0	Итоговый релиз проекта. Включает финальную сборку PollosFlow, backend-сервисы, frontend-модули, POS-эмулятор, банковский эмулятор, базу данных и миграции, API-документацию, тестовые данные, Risk List, результаты тестирования и демонстрационный сценарий.	2026-11-13

4.2.4 Project Schedule (Штатное расписание)

Роль	Ставка (для расчета стоимости реализации)
Руководитель проекта	4 600 руб./ч
Backend-разработчик	3 800 руб./ч
DevOps-инженер	3 700 руб./ч
Интеграционный разработчик	3 600 руб./ч
Системный аналитик	3 400 руб./ч
Frontend-разработчик	3 200 руб./ч
QA-инженер	3 000 руб./ч
UI/UX-ответственный	2 900 руб./ч

Ответственный за документацию	2 400 руб./ч
-------------------------------	--------------

Название задачи	Трудозатра ты Сени	Трудозатраты Доры	Трудозатр аты Пети	Общие трудозатр аты	Стоимость реализации
Фаза 1. Инициация проекта, итерация 1.1	8 ч	7 ч	5 ч	20 ч	68 600 руб.
Фаза 2. Анализ требований, итерация 2.1	12 ч	45 ч	10 ч	67 ч	230 700 руб.
Фаза 3. Проектирование архитектуры, итерация 3.1	35 ч	20 ч	20 ч	75 ч	278 500 руб.
Фаза 4. Реализация основных модулей, итерация 4.1	45 ч	25 ч	15 ч	85 ч	310 200 руб.
Фаза 4. Реализация основных модулей, итерация 4.2	35 ч	30 ч	20 ч	85 ч	306 000 руб.
Фаза 4. Реализация основных модулей, итерация 4.3	30 ч	25 ч	20 ч	75 ч	263 000 руб.

Фаза 5. Интеграция и сквозные сценарии, итерация 5.1	20 ч	8 ч	25 ч	53 ч	194 300 руб.
Фаза 5. Интеграция и сквозные сценарии, итерация 5.2	22 ч	8 ч	25 ч	55 ч	202 700 руб.
Фаза 6. Тестирование и стабилизация, итерация 6.1	15 ч	12 ч	20 ч	47 ч	162 100 руб.
Фаза 6. Тестирование и стабилизация, итерация 6.2	18 ч	5 ч	15 ч	38 ч	138 400 руб.
Фаза 7. Подготовка итогового релиза, итерация 7.1	10 ч	5 ч	5 ч	20 ч	73 000 руб.
Итого	250 ч	190 ч	180 ч	620 ч	2 227 500 руб.

4.2.5 *Project Resourcing (Ресурсы проекта)*

4.2.5.1 Staffing Plan (План по сотрудникам)

Роль	Количество специалистов	Требуемые навыки и опыт	Фазы участия
------	-------------------------	-------------------------	--------------

Руководитель проекта	1	Планирование работ, контроль сроков, управление задачами, координация команды, управление рисками, контроль состава релиза, коммуникация с заказчиком и проверяющими	Все фазы проекта
Системный аналитик	1	Анализ требований, подготовка SRS, описание пользовательских ролей, use-case диаграмм, ограничений, зависимостей и сценариев взаимодействия	Инициация, анализ требований, проектирование архитектуры, тестирование, подготовка итоговой документации
Backend-разработчик	1–2	Проектирование и реализация серверной логики, REST API, работа с PostgreSQL, Redis, RabbitMQ, Keycloak, реализация модулей партий средств, baseline, планирования, отчетности, аудита и сверки	Проектирование архитектуры, реализация, интеграция, тестирование, стабилизация
Frontend-разработчик	1	Разработка MFE-модулей, реализация интерфейсов для владельца, администратора, бухгалтера, кассира и авторизованного пользователя, работа с формами, таблицами, дашбордами, фильтрами, уведомлениями и валидацией	Проектирование архитектуры, реализация, тестирование, стабилизация
UI/UX-ответственный	1	Проектирование пользовательских экранов, применение UI-паттернов, обеспечение единообразия интерфейса, проектирование визуальной обратной связи, статусов, сообщений об ошибках и адаптивности интерфейса	Анализ требований, проектирование архитектуры, реализация frontend-модулей, тестирование

DevOps-инженер	1	Настройка окружения разработки и тестирования, запуск микросервисов, настройка PostgreSQL, RabbitMQ, Redis, Keycloak, Logstash, Sentry, конфигурация стендов, контроль логирования и мониторинга	Проектирование архитектуры, реализация, интеграция, тестирование, подготовка итогового релиза
QA-инженер	1	Подготовка тест-кейсов, функциональное, интеграционное, ролевое и регрессионное тестирование, фиксация дефектов, проверка соответствия требованиям SRS	Анализ требований, интеграция, тестирование, стабилизация, подготовка итогового релиза
Интеграционный разработчик	1	Разработка и проверка POS-эмулятора и банковского эмулятора, согласование API-контрактов, тестирование RabbitMQ, retry, timeout, fallback-сценариев, проверка сквозной обработки данных	Проектирование архитектуры, реализация, интеграция, тестирование
Ответственный за документацию	1	Поддержка актуальности SDP, SRS, use-case диаграмм, Risk List, фиксация изменений, согласование проектной документации и подготовка итогового комплекта материалов	Все фазы проекта

План участия ролей по фазам и итерациям

Фаза	Итерация	Период	Необходимые роли	Основные задачи
Фаза 1. Инициация проекта	1.1	2026-02-09 — 2026-02-11	Руководитель проекта, системный аналитик, ответственный за документацию	Определение цели проекта, области применения, ограничений, состава поставляемых артефактов и начальной версии SDP

Фаза 2. Анализ требований	2.1	2026-02-12 — 2026-02-27	Руководитель проекта, системный аналитик, UI/UX-ответственный, QA-инженер, ответственный за документацию	Подготовка SRS, описание ролей, функциональных и нефункциональных требований, use-case диаграмм и первичного Risk List
Фаза 3. Проектирование архитектуры	3.1	2026-03-02 — 2026-03-13	Руководитель проекта, backend-разработчик, frontend-разработчик, DevOps-инженер, интеграционный разработчик	Проектирование MSA, MFE, структуры БД, API-контрактов, интеграций с POS-эмулятором и банковским эмулятором
Фаза 4. Реализация основных модулей	4.1	2026-03-16 — 2026-03-27	Backend-разработчик, frontend-разработчик, DevOps-инженер, UI/UX-ответственный	Реализация базовых backend-сервисов, пользователей, ролей, авторизации, начальных frontend-модулей и структуры интерфейса
Фаза 4. Реализация основных модулей	4.2	2026-03-30 — 2026-04-10	Backend-разработчик, frontend-разработчик, UI/UX-ответственный	Реализация партий средств, business baseline, планирования распределения и связанных пользовательских экранов
Фаза 4. Реализация основных модулей	4.3	2026-04-13 — 2026-04-24	Backend-разработчик, frontend-разработчик, QA-инженер	Реализация генерации кассовых операций, отчетности, аудита, статусов операций и доработка основных модулей
Фаза 5. Интеграция и сквозные сценарии	5.1	2026-04-27 — 2026-09-11	Backend-разработчик, интеграционный разработчик, DevOps-инженер, QA-инженер	Подключение POS-эмулятора, проверка передачи кассовых операций, закрытия смен и формирования Z-отчетов

Фаза 5. Интеграция и сквозные сценарии	5.2	2026-09-14 — 2026-09-25	Backend-разработчик, интеграционный разработчик, QA-инженер, DevOps-инженер	Подключение банковского эмулятора, проверка банковских выписок, сверки данных и сквозного сценария обработки
Фаза 6. Тестирование и стабилизация	6.1	2026-09-28 — 2026-10-09	QA-инженер, backend-разработчик, frontend-разработчик, DevOps-инженер	Функциональное, интеграционное и ролевое тестирование, проверка основных сценариев, фиксация дефектов
Фаза 6. Тестирование и стабилизация	6.2	2026-10-12 — 2026-10-23	QA-инженер, backend-разработчик, frontend-разработчик, DevOps-инженер	Регрессионное тестирование, исправление критических дефектов, стабилизация системы, проверка соответствия требованиям SRS
Фаза 7. Подготовка итогового релиза	7.1	2026-10-26 — 2026-11-13	Руководитель проекта, QA-инженер, ответственный за документацию, DevOps-инженер	Финальная сборка проекта, обновление документации, подготовка демонстрационного сценария и передача проекта к итоговой сдаче

Закрытие потребности в сотрудниках

Участник	Закрываемые проектные роли
Сеня	Руководитель проекта, backend-разработчик, DevOps-инженер
Дора	Системный аналитик, frontend-разработчик, UI/UX-ответственная, ответственный за документацию по требованиям и интерфейсам
Петя	Backend-разработчик, QA-инженер, интеграционный разработчик

4.2.5.2 Resource Acquisition Plan (План поиска сотрудников)

все слоны на месте!

Так как текущая команда покрывает ключевые компетенции проекта, отдельный активный поиск сотрудников на момент начала разработки не требуется. Однако в случае наступления кадровых (отчислили блин) или компетентностных (skill issue) рисков может потребоваться временное привлечение дополнительных специалистов.

Если потребуется привлечение дополнительных сотрудников, поиск будет выполняться в следующем порядке:

1. **Внутреннее перераспределение задач внутри команды.**
Сначала руководитель проекта оценивает, можно ли перераспределить задачи между текущими участниками без привлечения новых людей.
2. **Временное расширение зоны ответственности текущих участников.**
Если проблема ограничена отдельной итерацией, сотрудник со смежной компетенцией может временно подключиться к задаче. Например, Сеня может помочь с интеграционной частью, Дора — с проверкой пользовательских сценариев, Петя — с backend-задачами и тестированием.
3. **Привлечение внешнего специалиста на ограниченный объём работ (chatGPT родненький).**
Если компетенции внутри команды недостаточно, может быть привлечён внешний специалист на конкретную задачу.
4. **Поиск через профессиональные и учебные контакты (найдем восстановленца).**
В первую очередь рассматриваются специалисты из доверенного окружения команды, так как проект имеет закрытый характер и требует ограничения доступа к внутренней информации.
5. **Привлечение подрядчика (itmo.help, заплатим 2-3 курсу безработному).**
Если задачу невозможно закрыть силами команды и доверенных контактов, допускается привлечение подрядчика на ограниченный срок и строго определённый объём работ.

Требования к новым сотрудникам

Новый сотрудник может быть привлечен к проекту только при соблюдении следующих условий:

1. наличие компетенций, необходимых для конкретной задачи;
2. готовность работать с существующей архитектурой и документацией проекта;
3. соблюдение правил доступа к репозиторию, тестовому стенду и проектной документации;
4. понимание ограничений проекта и сроков текущей фазы;
5. возможность быстро включиться в работу без длительного онбординга;
6. согласование кандидатуры руководителем проекта.

Для новых сотрудников доступ к проекту предоставляется по принципу минимально необходимых прав. Это означает, что специалист получает доступ только к тем репозиториям, сервисам, документам и задачам, которые необходимы для выполнения его работы.

Процесс привлечения сотрудника

Процесс привлечения дополнительного сотрудника включает следующие этапы:

Этап	Описание	Ответственный
Выявление потребности	Определяется проблема, которую нельзя закрыть текущими ресурсами	Сеня
Описание роли	Формулируются требуемые навыки, объем работ и срок привлечения	Сеня, ответственный за область
Поиск кандидата	Рассматриваются внутренние и внешние источники поиска	Дора
Оценка кандидата	Проверяется опыт, компетенции и возможность быстро включиться в проект	Сеня, Дора или Петя
Подключение к проекту	Сотруднику выдаются задачи, документация и ограниченные доступы	Сеня
Контроль работы	Результаты работы проверяются через code review, тестирование или ревью документации	Ответственный за область
Завершение участия	После выполнения задачи доступы сотрудника закрываются, результаты фиксируются в документации	Сеня

4.2.6 Budget (Бюджет)

Статья бюджета	Минимальная стоимость	Ожидаемая стоимость	Максимальная стоимость	Комментарий
Работа команды разработки	1 780 500 руб.	2 227 500 руб.	2 887 000 руб.	Оплата трудозатрат Сени, Доры и Пети
Инфраструктура разработки и тестирования	80 000 руб.	180 000 руб.	350 000 руб.	Серверы dev/stage, БД, очереди, кэш, тестовые окружения, бэкапы
Инструменты разработки и командной работы	20 000 руб.	60 000 руб.	120 000 руб.	Git-хостинг, таск-трекер, UI/UX-инструменты, средства тестирования

ChatGPT Pro для команды	180 000 руб.	360 000 руб.	360 000 руб.	\$200/мес × 3 человека × 3/6/6 месяцев
Эмуляторы и тестовые интеграции	50 000 руб.	120 000 руб.	220 000 руб.	POS-эмулятор, банковский эмулятор, генератор baseline-данных
Тестирование и стабилизация	50 000 руб.	120 000 руб.	250 000 руб.	Функциональное, интеграционное, ролевое, нагрузочное и регрессионное тестирование
Документация и подготовка релиза	30 000 руб.	80 000 руб.	150 000 руб.	SRS, SDP, use-case диаграммы, Risk List, API-документация, демо-сценарий
Резерв на кадровые риски	80 000 руб.	150 000 руб.	200 000 руб.	Потеря участников команды, привлечение внешних специалистов и подрядчиков
Организационные риски	40 000 руб.	70 000 руб.	250 000 руб.	Смена ключевого стейкхолдера, изменение налоговых и регуляторных требований
Резерв на интеграционные риски	80 000 руб.	130 000 руб.	250 000 руб.	Недоступность банковских тестовых контуров, изменение спецификации POS-терминалов
Итого	2 390 500 руб.	3 497 500 руб.	5 037 000 руб.	Полный бюджет проекта

4.3 Project Monitoring and Control (Мониторинг и контроль проекта)

4.3.1 Requirements Management Plan (План управления требованиями)

Управление требованиями проекта PollosFlow осуществляется на основе SRS, use-case диаграмм и согласованных проектных решений. Все функциональные и нефункциональные требования фиксируются в SRS, а пользовательские сценарии описываются в отдельных use-case диаграммах.

Контроль требований включает следующие действия:

1. фиксация всех требований в SRS;
2. присвоение требованиям уникальных идентификаторов;
3. связывание требований с пользовательскими ролями и use-case диаграммы;
4. проверка реализуемости требований с технической точки зрения;
5. согласование изменений требований с руководителем проекта и заинтересованными сторонами;
6. обновление SDP, SRS и связанных документов при изменении требований;
7. проверка реализации требований на этапах тестирования и приемки.

Изменение требований допускается только после письменной фиксации причины изменения, оценки влияния на сроки, бюджет, архитектуру и состав релиза. Устные договоренности не считаются основанием для изменения требований до тех пор, пока они не внесены в проектную документацию.

Ответственность за управление требованиями распределяется следующим образом:

Участник	Ответственность
Сеня	Проверка технической реализуемости требований, оценка влияния изменений на backend, инфраструктуру, сроки и релиз
Дора	Основная ответственность за SRS, пользовательские роли, сценарии использования, UI/UX-требования и актуальность требований
Петя	Проверка требований с точки зрения тестирования, интеграций, POS-эмулятора, банковского эмулятора и сквозных сценариев

Требование считается контролируемым, если оно зафиксировано в SRS, связано с задачами реализации, имеет критерии проверки и может быть подтверждено тестированием.

4.3.2 Schedule Control Plan (План управления расписанием)

Контроль расписания проекта осуществляется на основе плана фаз и задач в таск-трекере. Расписание проекта включает семь фаз: инициацию, анализ требований, проектирование архитектуры, реализацию основных модулей, интеграцию, тестирование и подготовку итогового релиза. Для каждой фазы определены даты начала и окончания, а также количество итераций.

Контроль сроков выполняется следующими способами:

1. каждая задача фиксируется в таск-трекере с ответственным исполнителем и сроком выполнения;

2. в начале каждой итерации определяется набор задач, которые должны быть выполнены;
3. в конце каждой итерации проводится проверка достигнутых результатов;
4. руководитель проекта отслеживает отклонения от плана;
5. при возникновении задержек задачи перераспределяются или переносятся в следующую итерацию;
6. критические задержки фиксируются как проектные риски;
7. изменения сроков отражаются в SDP и согласуются с командой.

Контроль сроков выполняется не только по факту завершения фаз, но и по промежуточным результатам. Для каждой итерации должны быть определены ожидаемые артефакты: документ, модуль, API-контракт, экран интерфейса, тест-кейс, эмулятор или результат тестирования.

Объект контроля	Способ контроля	Ответственный
Фазы проекта	Проверка дат начала и завершения фаз	Сеня
Итерации	Проверка выполнения задач внутри итерации	Сеня
Требования и документация	Проверка актуальности SRS, use-case диаграммы и SDP	Дора
Backend и инфраструктура	Проверка готовности сервисов, API, БД и окружения	Сеня
Frontend	Проверка готовности экранов и пользовательских сценариев	Дора
Интеграции и тестирование	Проверка POS-эмулятора, банковского эмулятора, тест-кейсов и дефектов	Петя

Если задача не может быть завершена в рамках текущей итерации, руководитель проекта определяет один из вариантов действий: уменьшение объема задачи, перенос части работ в следующую итерацию, перераспределение исполнителей или изменение приоритета задач.

4.3.3 Budget Control Plan (План управления бюджетом)

Контроль бюджета проекта выполняется на основе оценки трудозатрат и почасовых ставок участников команды. В SDP уже определены минимальные, ожидаемые и максимальные трудозатраты по проекту: 495 часов, 620 часов и 815 часов соответственно. Также рассчитана стоимость проекта по участникам и общая стоимость разработки.

Основной способ контроля бюджета - сопоставление плановых трудозатрат с фактически выполненными работами. Для каждой задачи в таск-трекере фиксируется исполнитель, плановая оценка времени и фактически затраченное время.

Контроль бюджета включает:

1. учет плановых трудозатрат по каждой фазе и итерации;
2. учет фактических трудозатрат участников;
3. сравнение фактических затрат с ожидаемой оценкой;

4. выявление перерасхода времени по задачам;
5. анализ причин перерасхода;
6. корректировку объема работ при превышении бюджета;
7. пересмотр оценки при изменении требований или состава релиза.

Порогом внимания считается превышение плановых трудозатрат по задаче более чем на 20%. В этом случае задача анализируется отдельно: определяется причина отклонения, влияние на общий бюджет и необходимость изменения плана.

Порогом критического отклонения считается превышение ожидаемой стоимости проекта более чем на 30%. В этом случае руководитель проекта пересматривает состав релиза, приоритеты задач и возможность переноса части функциональности за пределы текущего релиза.

4.3.4 Quality Control Plan (План управления качеством)

Контроль качества проекта PollosFlow направлен на проверку соответствия реализованной системы требованиям SRS, корректность работы основных сценариев, надежность интеграций, безопасность доступа и стабильность работы системы.

Качество контролируется на нескольких уровнях:

1. проверка требований и проектной документации;
2. код-ревью изменений;
3. функциональное тестирование;
4. интеграционное тестирование;
5. тестирование ролевой модели доступа;
6. проверка UI/UX и пользовательских сценариев;
7. регрессионное тестирование после исправления дефектов;
8. проверка демонстрационного сценария перед итоговым релизом.

Основные критерии качества:

Критерий	Способ проверки	Ответственный
Соответствие требованиям SRS	Проверка реализации требований и use-case диаграммы	Дора, Петя
Корректность backend-логики	Юнит-тесты, код-ревью, проверка API	Сеня
Корректность frontend-интерфейсов	Проверка экранов, форм, таблиц, фильтров, уведомлений	Дора
Работа ролевой модели	Проверка доступа для владельца, администратора, бухгалтера, кассира и пользователя	Петя
Работа POS-интеграции	Проверка передачи операций, закрытия смен и Z-отчетов	Петя

Работа банковской интеграции	Проверка выписок, сверки, расхождений и fallback-сценариев	Петя
Стабильность окружения	Проверка запуска сервисов, логирования и инфраструктуры	Сеня
Готовность к релизу	Финальная проверка сборки и демонстрационного сценария	Сеня, Дора, Петя

Дефекты фиксируются в таск-трекере с указанием описания проблемы, шага воспроизведения, ожидаемого результата, фактического результата, приоритета и ответственного исполнителя.

Дефекты классифицируются по критичности:

Уровень	Описание
Critical	Ошибка блокирует основной сценарий работы системы или делает невозможной демонстрацию проекта
Major	Ошибка нарушает важную функциональность, но имеет обходной путь
Minor	Ошибка не нарушает основной сценарий, но ухудшает удобство или корректность работы
Trivial	Небольшая визуальная, текстовая или техническая проблема

Перед подготовкой итогового релиза не должно оставаться открытых дефектов уровня Critical и Major. Дефекты уровня Minor и Trivial могут быть перенесены в список известных ограничений, если они не влияют на демонстрационный сценарий и основные требования проекта.

4.3.5 Reporting Plan (План отчетности)

Так как разработка организована итерационно, основная отчетность формируется один раз в три недели по итогам спринта.

В проекте используются следующие виды отчетности:

Вид отчета	Периодичность	Содержание	Ответственный
Отчет по итогам спринта	1 раз в 3 недели	Выполненные задачи, задачи в работе, невыполненные задачи, созданные артефакты, проблемы, риски, отклонения по срокам и бюджету	Сеня

Отчет по требованиям	После изменения требований или завершения фазы анализа требований	Измененные требования, причина изменения, влияние на сроки, бюджет, архитектуру и состав релиза	Дора
Отчет по тестированию	На фазе тестирования и стабилизации, а также по итогам тестовых прогонов	Проверенные сценарии, найденные дефекты, критичность дефектов, статус исправлений, результат регрессионного тестирования	Петя
Отчет по релизу	Перед итоговой сдачей проекта	Состав релиза, реализованные модули, известные ограничения, результаты тестирования, готовность документации и демонстрационного сценария	Сеня, Дора, Петя

Отчет по итогам спринта должен включать:

1. дату отчета;
2. номер спринта;
3. период спринта;
4. цель спринта;
5. выполненные задачи;
6. задачи, находящиеся в работе;
7. задачи, перенесенные в следующий спринт;
8. созданные или обновленные артефакты;
9. выявленные проблемы;
10. новые или изменившиеся риски;
11. отклонения по срокам и бюджету;
12. план работ на следующий спринт.

По итогам каждого трехнедельного спринта руководитель проекта оценивает, достигнута ли цель спринта. Если цель спринта не достигнута, в отчете фиксируются причины отклонения и выбранное решение: перенос задачи, уменьшение объема работ, перераспределение исполнителей, изменение приоритетов или корректировка плана следующего спринта.

Отчет по спринту используется для контроля соответствия фактического прогресса фазовому плану проекта. Если по итогам спринта выявлено, что сроки фазы находятся под угрозой, руководитель проекта фиксирует отклонение и выносит его на обсуждение команды.

Итоговый отчет по проекту формируется перед сдачей и включает состав итогового релиза, перечень реализованных функций, результаты тестирования, список известных ограничений, состояние документации и готовность демонстрационного сценария.

4.4 Risk Management plan (План управления рисками)

Управление рисками проекта осуществляется на основе отдельного документа [Risk List](#), в котором зафиксированы основные риски проекта, их вероятность, критичность, итоговый уровень риска, признаки проявления, стратегия смягчения и стратегия действий при наступлении риска. Документ Risk List используется для отслеживания, приоритизации и управления рисками на всех этапах жизненного цикла проекта.

Контроль рисков выполняется на протяжении всех фаз разработки проекта и направлен на своевременное выявление отклонений, которые могут повлиять на сроки, качество, бюджет, состав релиза или возможность итоговой сдачи проекта.

Для оценки вероятности и влияния используется шкала от 1 до 5. Итоговый уровень риска определяется по матрице пересечения вероятности и влияния. Уровни риска дополнительно обозначаются цветами: синий — очень низкий, зелёный — низкий, жёлтый — средний, оранжевый — высокий, красный — очень высокий.

Контроль рисков включает следующие действия:

1. выявление рисков на фазах анализа, проектирования, реализации, интеграции, тестирования и подготовки релиза;
2. фиксация риска в Risk List;
3. определение вероятности, влияния и итогового уровня риска;
4. назначение ответственного за контроль риска;
5. отслеживание численных метрик риска;
6. применение стратегии смягчения при переходе риска в состояние предупреждения;
7. применение contingency plan при переходе риска в состояние тревоги;
8. пересмотр сроков, состава работ или состава релиза при наступлении критических рисков;
9. фиксация изменений и наступивших рисков в отчетах по спринтам.

Реестр рисков пересматривается по итогам каждого трехнедельного спринта. Дополнительно риск может быть рассмотрен внепланово, если одна из его метрик переходит в состояние предупреждения или тревоги.

Для каждого риска используются три состояния метрик:

Состояние	Значение
Норма	Риск находится в допустимом состоянии и не требует дополнительных действий, кроме регулярного наблюдения
Предупреждение	Риск требует применения стратегии смягчения и дополнительного контроля
Тревога	Риск считается наступившим или близким к наступлению; активируем стратегию наступления

При переходе хотя бы одной метрики риска в состояние предупреждения ответственный участник применяет стратегию смягчения риска. При переходе хотя бы одной метрики риска в состояние

тревоги риск переводится в статус проблемы, для него назначается ответственный, срок устранения и конкретный план действий.

Распределение ответственности за риски

Участник	Зона ответственности
Сеня	Контроль рисков сроков, архитектуры, backend-разработки, инфраструктуры, межсервисных контрактов и итоговой сборки проекта
Дора	Контроль рисков требований, SRS, пользовательских сценариев, UI/UX, документации и согласований с заказчиком
Петя	Контроль рисков интеграций, POS-эмулятора, банковского эмулятора, тестирования, дефектов и сквозных сценариев

Действия при наступлении риска

Если риск наступил, выполняется следующий порядок действий:

1. риск переводится в статус проблемы;
2. фиксируется дата наступления риска;
3. определяется затронутая фаза, итерация, модуль или артефакт;
4. назначается ответственный за устранение последствий;
5. выбирается и активируется план наступления из Risk List;
6. оценивается влияние риска на сроки, бюджет, качество и состав релиза;
7. при необходимости корректируется план работ;
8. информация о наступившем риске включается в отчет по итогам спринта.

Если наступивший риск влияет на сроки или состав релиза, руководитель проекта инициирует пересмотр плана работ и фиксирует изменения в SDP. Если риск влияет на требования, Дора актуализирует SRS и связанные use-case. Если риск влияет на интеграции или тестирование, Петя актуализирует тест-кейсы и план проверки сквозных сценариев.

Наибольшее влияние на план проекта оказывают следующие риски из [Risk List](#):

R-02. Отсутствие или нестабильность тестовых контуров банковских интеграций — напрямую влияет на фазу 5 «Интеграция и сквозные сценарии»: при недоступности банковского тестового контура сдвигаются релизы 0.6–0.7 и срок начала тестирования. Поэтому на фазу 5 заложен наибольший календарный интервал, а интеграционные работы начинаются с эмуляторов, не зависящих от внешних сервисов.

R-05, R-08. Потеря участников команды и трудности привлечения новых специалистов — влияют на штатное расписание (раздел 4.2.4) и сроки всех фаз: выбытие исполнителя роли требует перераспределения трудозатрат между оставшимися участниками. Влияние компенсируется планом поиска сотрудников (раздел 4.2.5.2) и резервом на риски в бюджете проекта (раздел 4.2.6).

4.5 Close-out Plan (План завершения проекта)

Завершение проекта PollosFlow определяется достижением целей разработки, готовностью итогового релиза, завершением тестирования, актуальностью проектной документации и готовностью системы к итоговой сдаче.

Проект считается завершенным, если выполнены следующие условия:

1. реализованы основные функции системы PollosFlow;
2. подготовлены и проверены POS-эмулятор и банковский эмулятор;
3. реализован основной сквозной сценарий обработки данных;
4. выполнено функциональное, интеграционное, ролевое и регрессионное тестирование;
5. исправлены все открытые дефекты уровней Critical и Major;
6. оставшиеся дефекты уровней Minor и Trivial, если они есть, не блокируют основной демонстрационный сценарий и зафиксированы в списке известных ограничений;
7. актуализирована вся проектная документация;
8. зафиксированы известные ограничения проекта;
9. подготовлен демонстрационный сценарий;
10. сформирован итоговый релиз проекта;
11. проект готов к передаче на итоговую сдачу.

Отсутствие открытых дефектов уровней Critical и Major является обязательным условием закрытия проекта. Дефект уровня Critical считается блокирующим, если он делает невозможным запуск системы, выполнение основного сквозного сценария, работу ключевых модулей или демонстрацию проекта. Дефект уровня Major считается значимым, если он нарушает работу важной функциональности, ролевой модели доступа, POS-интеграции, банковской интеграции, отчетности или сверки данных.

Дефекты уровней Minor и Trivial допускаются только в том случае, если они не влияют на основной сквозной сценарий, безопасность доступа, корректность интеграций, итоговую демонстрацию и выполнение ключевых требований SRS. Такие дефекты должны быть зафиксированы в списке известных ограничений проекта.

Критерии завершения разработки

Область	Критерий завершения	Ответственный
Функциональность	Реализованы ключевые модули: управление пользователями, роли, партии средств, baseline, планирование распределения, отчетность и сверка	Сеня, Дора
Backend	Backend-сервисы запускаются, API доступны, основные сценарии обработки данных работают корректно	Сеня
Frontend	Реализованы основные интерфейсы для пользовательских ролей, формы, таблицы, статусы, уведомления и базовые дашборды	Дора

POS-эмулятор	Проверена передача операций, закрытие смен и формирование Z-отчетов	Петя
Банковский эмулятор	Проверено получение выписок, сверка данных и обработка расхождений	Петя
Сквозной сценарий	Проверена цепочка обработки данных между основными этапами системы	Сеня, Петя
Тестирование	Выполнены основные тестовые сценарии, проведено регрессионное тестирование	Петя
Дефекты	Отсутствуют открытые дефекты уровней Critical и Major; дефекты Minor и Trivial либо исправлены, либо зафиксированы как известные ограничения	Петя, Сеня
Риски	Risk List актуализирован, наступившие риски обработаны, открытые критические риски зафиксированы как ограничения	Сеня
Документация	Обновлены SDP, SRS, use-case диаграммы, Risk List и итоговые материалы	Дора, Сеня
Релиз	Подготовлена финальная сборка и демонстрационный сценарий	Сеня

Действия перед завершением проекта

Перед закрытием проекта выполняются следующие работы:

1. финальная проверка состава релиза;
2. проверка запуска всех сервисов и эмуляторов;
3. проверка основного демонстрационного сценария;
4. проверка соответствия реализации требованиям SRS;
5. финальное регрессионное тестирование;
6. проверка отсутствия открытых дефектов уровней Critical и Major;
7. анализ оставшихся дефектов уровней Minor и Trivial;
8. фиксация допустимых оставшихся дефектов в списке известных ограничений;
9. актуализация проектной документации;
10. актуализация Risk List и фиксация состояния рисков;
11. подготовка итогового отчета по проекту;
12. передача проекта к итоговой сдаче.

Итоговый релиз

Итоговый релиз проекта должен включать:

1. исходный код системы PollosFlow;

2. backend-сервисы;
3. frontend-модули;
4. POS-эмулятор;
5. банковский эмулятор;
6. базу данных и миграции;
7. конфигурации запуска;
8. API-документацию;
9. тестовые данные;
10. проектную документацию;
11. Risk List с актуальным состоянием рисков;
12. результаты тестирования;
13. список известных ограничений;
14. демонстрационный сценарий.

Действия после завершения разработки

После завершения разработки команда фиксирует итоговое состояние проекта. В репозитории создается финальный тег или релизная ветка, документация приводится в актуальное состояние, а список известных ограничений сохраняется вместе с итоговыми материалами проекта.

После закрытия проекта должны быть выполнены следующие действия:

1. сохранение итоговой версии исходного кода;
2. сохранение итоговой версии документации;
3. фиксация состава релиза;
4. архивирование тестовых данных и результатов тестирования;
5. архивирование списка дефектов и статуса их исправления;
6. фиксация известных ограничений и возможных направлений дальнейшего развития;
7. подготовка краткого отчета о достигнутых результатах;
8. передача проекта на итоговую оценку.

Проект считается закрытым после подтверждения готовности итогового релиза, успешной демонстрации основного сценария, отсутствия открытых дефектов уровней Critical и Major, актуализации Risk List и передачи комплекта проектных материалов. Финальное закрытие выполняет руководитель проекта после согласования с участниками команды.